KALYAN COVERING STORE • TECHNICAL MASTER SPECIFICATION

Enterprise Full-Stack Architecture Blueprint

Complete Reference for Human Developers & AI Agents (Directory, API, Data Schema, Razorpay, Security)

Document Version: 2.0 | Date: 26 Oct 2026 | Stack: Vanilla Vite + Express Vercel + Firebase Firestore

MASTER ARCHITECTURAL SPECIFICATION

1. Executive Architecture & Core Philosophy

Kalyan Covering is an Indian high-trust jewelry e-commerce platform specializing in 1-gram gold-plated micro-covering ornaments, bridal sets, rings, and bangles. The application architecture follows a decoupled, high-performance JAMstack design: a lightweight Vanilla JS ES6+ Single Page Application (SPA) bundled via Vite, hosted on Firebase/Vercel Hosting, backed by serverless Express microservices running on Node.js/Vercel Serverless Functions, with Google Cloud Firestore serving as the authoritative transactional database.

Architectural Invariants (Non-Negotiable Rules)

• Zero Client-Side Pricing Trust: The frontend NEVER dictates product prices, discounts, or delivery fees. All pricing is recalculated and validated server-side in /api/payments and /api/orders using Firestore as single-source-of-truth.

• Two-Phase Payment Commitment: Order insertion requires cryptographic HMAC SHA256 signature verification before any order status changes to PAID.

• Decoupled Modular Styling: Vanilla CSS utilizing CSS Custom Properties in design-system.css and components.css. Tailwind or monolithic CSS frameworks are NOT used to maintain fast load times on Indian mobile networks.

2. System Topology & Request Lifecycle

END-TO-END SYSTEM TOPOLOGY

Browser Layer: Mobile/Desktop SPA (public/customer/*, public/admin/dashboard.html) -> Vanilla JS Services

CDN & Assets: Cloudinary CDN (Product Images, Videos, Reviews) + Static Hosting (Vite / Firebase)

Gateway Layer: Vercel Serverless API (/api/index.js) -> Express Routers (/server/routes/*)

Security Layer: Bearer Token Auth (Firebase Admin SDK), RBAC Middleware, Rate Limiting (express-rate-limit)

Database & Auth: Cloud Firestore (Collections: products, orders, users, coupons, settings) + Firebase Auth

Partner Logistics: Shiprocket Shipping API (Automated AWB, Pincode Verification, Tracking Updates)

Payment Subsystem: Razorpay Standard Checkout + Server HMAC SHA256 Verification + Webhooks

3. Codebase Directory Structure & Responsibilities

Any new AI or engineer working on this repository must strictly adhere to the established folder separation to avoid introducing regressions.

DIRECTORY MAP & PURPOSE

```
public/                 Frontend root bundled by Vite
  |-- customer/         Storefront pages (index, products, product-detail, cart, checkout, auth, my-orders)
  |-- admin/            Admin portal (admin-login.html, 400KB dashboard.html full-featured admin SPA)
  |-- css/              Design system: design-system.css, components.css, customer.css, admin.css
  |-- js/               Modular ES6 client-side JavaScript architecture
      |-- components/   Reusable UI components (Navbar.js, Footer.js, ProductCard.js, Modal.js, Toast.js)
      |-- services/     Data access layer (authService, productService, cartService, orderService, paymentService)
      |-- store/        Reactive client state manager (state.js: cart, wishlist, auth sync)
      |-- utils/        Utility helpers (cloudinaryUtils.js, validation.js, helpers.js)
      |-- firebase-config.js  Firebase SDK client initialization, AppCheck config, callVercelApi helper
      |-- rbac.js       Client-side role rendering (Customer, Staff, Manager, Owner)
api/index.js            Vercel entrypoint delegating all /api/* requests to Express server
server/                 Backend server code
  |-- lib/              admin.js (Firebase Admin SDK), stock.js (Atomic inventory locks), utils.js
  |-- middleware/       auth.js (verifyIdToken, requireAdmin), rateLimit.js (DDoS prevention)
  |-- routes/           Express routers (payments, orders, products, shipping, admin, invoices, uploads)
scripts/                Maintenance CLI scripts (check-razorpay-keys.js, generate-architecture-pdf.js)
firestore.rules         Production Firestore security rules governing client vs server permissions
```

4. Database Schema & Collections (Google Cloud Firestore)

All data resides in Firestore document collections with strict schemas:

• products: { id, name, price, originalPrice, category, subcategory, stock, images: [], videos: [], specifications: {}, rating, reviewCount, status: "active"|"draft", createdAt }

• orders: { orderId, uid, items: [{ productId, name, price, qty, img }], pricing: { subtotal, discount, shipping, tax, total }, address: { fname, lname, phone, address, city, state, pin }, payment: { method: "razorpay"|"cod", status: "PENDING"|"PAID", razorpayOrderId, razorpayPaymentId }, shipping: { awb, courier, status: "ORDER_PLACED"|"SHIPPED"|"DELIVERED" }, createdAt }

• users: { uid, email, displayName, phone, role: "customer"|"staff"|"manager"|"owner", addresses: [], defaultAddressId, createdAt }

• coupons: { code: "SUMMER10", type: "percent"|"fixed", value: 10, minOrder: 500, maxDiscount: 200, status: "active", expiresAt }

• audit_logs: { timestamp, actorUid, actorEmail, action, resource, details: {} }

MASTER ARCHITECTURAL SPECIFICATION

5. Razorpay Payment Architecture & Cryptographic Verification

Online payments use Razorpay's Standard Checkout modal backed by server-side order initiation and HMAC SHA256 cryptographic verification.

Detailed Payment Sequence

• 1. Pre-Payment Validation: Customer clicks "Place Order" in checkout.html. The client posts items, coupon, and address to POST /api/payments/create-order.

• 2. Authoritative Price Resolution: The server loads each item from Firestore products collection. It multiplies dbPrice * qty, applies verified coupon discounts from coupons collection, checks pin code serviceability, and computes totalAmount in Paise (INR * 100).

• 3. Razorpay Order Creation: Server calls Razorpay SDK: rzp.orders.create({ amount: totalAmountInPaise, currency: "INR", receipt: orderId, notes: { uid, orderId } }). The server saves a PENDING order in Firestore and returns the Razorpay orderId to the browser.

• 4. Client Checkout Modal: Frontend initializes window.Razorpay with key_id, orderId, customer prefill info, and handler callback.

• 5. Signature Verification: Upon payment completion, Razorpay returns: razorpay_order_id, razorpay_payment_id, and razorpay_signature. Client immediately submits them to POST /api/payments/verify.

• 6. Cryptographic HMAC Check: Server computes: expectedSignature = crypto.createHmac("sha256", process.env.RAZORPAY_KEY_SECRET).update(`${orderId}|${paymentId}`).digest("hex"). Only if expectedSignature === signature does the server mark order as PAID, decrement inventory atomically via stock.js, and trigger email invoice.

Testing & Sandbox Verification

• Test Key: rzp_test_... (Check validity using: npm run check:razorpay)

• Test Card: 4111 1111 1111 1111 | Expiry: Any Future | CVV: 123 | OTP: 123456

• Test UPI: success@razorpay (Simulates instantaneous UPI authorization)

6. Critical Gotchas & Bug Audit References

A full audit was conducted in BUG_REPORT.md. Any engineer or AI modifying code must be aware of:

• API Timeout: public/js/firebase-config.js line 101 has a 30s AbortController timeout. Must be adjusted to 60s for slow Indian 4G/3G connections.

• Quick View Modal: components.css defines #quick-view-modal but the HTML markup is missing from customer pages. Must be injected via DOM.

• Size Guide Modal: product-detail.html uses modal.classList.add("on"), whereas CSS listens to .modal.open.

MASTER ARCHITECTURAL SPECIFICATION

7. Production Standards & AI Developer Handover

This section provides the master prompt and operational rules for any new AI model tasked with maintaining, extending, or fixing Kalyan Covering.

AI ONBOARDING PROMPT & SYSTEM CONTEXT

AI CONTEXT DIRECTIVE:

"You are the lead full-stack engineer for Kalyan Covering (https://kalyancoveringstore-c53e4.web.app).

Repository Structure: Vanilla HTML/JS frontend in public/, Express backend in server/, DB is Cloud Firestore.

Core Rules:

1. Maintain Vanilla CSS (design-system.css, components.css). Do NOT introduce Tailwind or large frameworks.
2. Never trust client prices: all financial calculations must be handled in server/routes/payments.js.
3. Preserve DOM IDs and existing class conventions (.modal.open, .toast, .btn-primary).
4. All sensitive operations (inventory updates, order cancellations, refunds) must run through server APIs.
5. Adhere to BUG_REPORT.md and implementation_plan.md for all pending fixes."

Production Launch Checklist

• Mandatory Policy Pages: Privacy Policy, Terms of Service, Shipping & Delivery Policy, Return & Refund Policy (Required for Razorpay Live Gateway Approval).

• Contact Information: Display GSTIN number, physical store address, customer service phone, and grievance officer email.

• Webhooks Implementation: Configure POST /api/payments/webhook with Razorpay Webhook Secret on Vercel to capture dropped client sessions.

• SEO & OpenGraph: Implement meta tags on customer pages for dynamic WhatsApp / Facebook sharing previews with image and price.

• Monitoring & Logs: Configure Sentry for client/server error tracking and SonarLint for ongoing static analysis.